

Linux Kernel Fuzzing with Syzkaller

Leonardo Scoppitto - 545615

Contents

1	Introduction to Fuzzing	1
1.1	Kernel Fuzzing	1
2	Syzkaller	3
2.1	Syzlang Grammar	4
2.2	Fuzzing process	4
3	Project Setup	5
3.1	Configuration	5
3.1.1	Buildroot	6
4	Injecting Kernel Bugs	8
4.1	Custom Syscalls	8
4.2	Kernel patches	9
4.3	DVKM	10
5	Demonstration of Findings	13
5.1	Reading reproduction outputs	14
5.1.1	KASAN output	18
5.1.2	UBSAN output	21

Fuzzing is an automated software testing technique that involves providing invalid, unexpected, or random data as inputs to a computer program. The program is then monitored for exceptions such as crashes, failing assertions or potential memory leaks. The primary objective of fuzzing is to discover software vulnerabilities and robustness defects that traditional, deterministic testing methodologies might fail to find.

A fuzzer acts as a pseudo-random input generator coupled with an execution monitor: when the target application processes the malformed input and enters an invalid state, the execution monitor catches the resulting fault and the input that triggered the fault is then saved for further analysis.

Fuzzers can be categorized based on how they generate test cases:

- **Mutation-based** fuzzers take an existing corpus of valid inputs and apply structural mutations to them.
- **Generation-based** fuzzers construct inputs from scratch using a predefined grammar or template that describes the input format.

Fuzzers are also classified based whether they can have a view on the original source code or not:

- **Black-box** fuzzing operates with no internal knowledge of the target application structure or code logic, treating the binary as an opaque entity and monitoring only outputs, terminations and how the system/program responds to the inputs
- **White-box** fuzzing leverages program analysis techniques systematically analyze the internal control-flow graph, thus being able to reach a higher code coverage.
- **Grey-box** fuzzing is a middle ground between black and white box fuzzing as it exploits data structures and tools injected during compilation to understand whether an input reached a given state or managed to get into a new branch of the execution.

1.1 Kernel Fuzzing

When a user-space application processes an input, it starts from a clean environment and if it crashes, the operating system cleans up the memory and the fuzzer restarts the process. On the other hand, the kernel is stateful and if a syscall executed by the fuzzer alters the global state of the system, subsequent syscalls operate on the modified state, meaning that a bug could arise only after a specific chain of system calls. Moreover, a crash that makes the kernel panic destroys the testing environment, so the fuzzer cannot just restart the target process, but it has to reboot virtual machine.

A fuzzer can only detect fatal errors and not, for example, errors that lead to a memory corruption but not to a crash. For this reason, the kernel has to be compiled with the correct flags that allow the fuzzer to understand when an execution leads to a corrupted state. **Kernel Address Sanitizer** (KASAN) allows to discover memory errors such as *out-of-bounds* and *use-after-free* bugs by allocating one byte of *shadow memory* every eight bytes of kernel memory, allowing to track the allocation state of the original bytes. The compiler then inserts validation checks before every memory read and write operations so that if a fuzzer mutates an input that causes the kernel to read past the end of an allocated buffer, the kernel panics and a report is generated.

For coverage feedback, the Linux kernel provide built-in instrumentation mechanisms such as **KCOV**, which exposes kernel code coverage information via debugfs under `/sys/kernel/debug/kcov`. To further enhance the feedback loop, the kernel instrumentation can be extended via the `KCOV_ENABLE_COMPARISONS` feature, which enables the compiler to instrument comparison instructions and switch statements throughout the kernel code to intercept their operands during runtime, allowing the fuzzer to extract those operands and use them as mutation hints.

[Syzkaller](#) is an unsupervised coverage-guided kernel fuzzer supporting FreeBSD, Fuchsia, gVisor, Linux, NetBSD, OpenBSD and Windows. Its architecture consist in two components:

- the **manager**, which runs on a host machine;
- the **executor**, which performs the fuzzing inside a virtual machine using tools like QEMU and KVM.

The manager and the executor communicate via SSH for control commands and via remote procedure calls (RPC) for data synchronization.

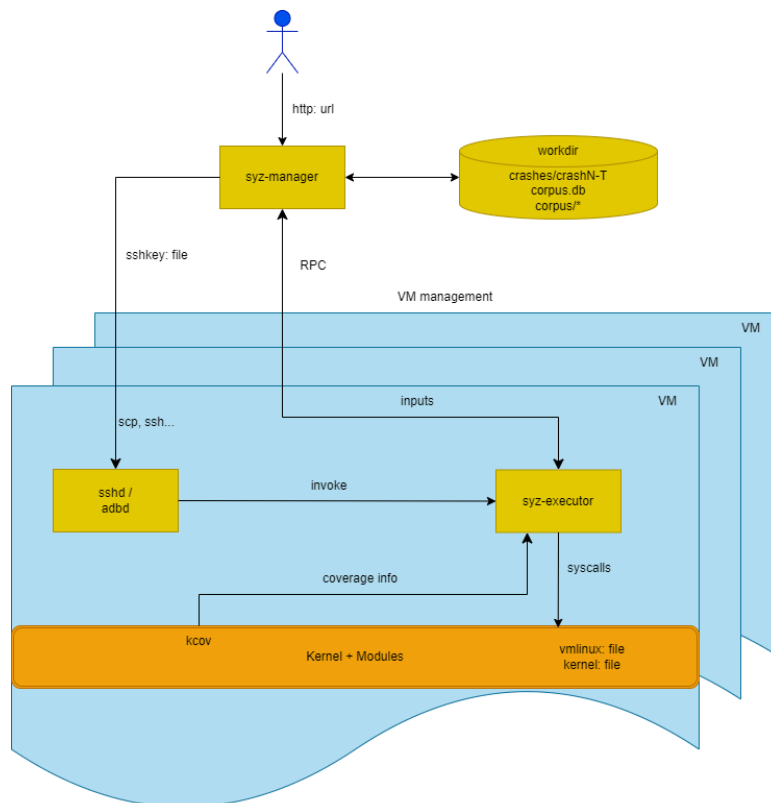


Figure 1: Syzkaller architecture

syz-manager

The `syz-manager` process is run on the host operating system and supervises the testing environment, generates new inputs and mutations and also instantiates virtual machine targets based on parameters defined in the configuration file, specifying memory allocations and CPU core counts.

When a virtual machine becomes unresponsive due to a kernel hang or a panic, `syz-manager` detects the failure by parsing the virtual serial console output stream, archives the raw logs, initiates a hard reboot, and provisions a new worker process to resume testing.

The manager also maintains the persistent master corpus containing all discovered test inputs that yield an higher code coverage and serves the analytics dashboard where the user can check the fuzzing process and the bugs found.

syz-executor

The syz-executor is a stateless execution engine deployed inside each virtual machine instance. It communicates directly with the host-side syz-manager over an RPC connection to receive the generated system call programs and report back execution results, including error statuses and collected code coverage metrics.

To execute programs safely, syz-executor spawns transient subprocesses, each responsible for executing a single input program (a sequence of syscalls). It is designed to be as simple as possible to avoid interfering with the core fuzzing environment. The executor is written in C++, compiled as a static binary, and utilizes shared memory regions for fast communication the subprocesses.

2.1 Syzlang Grammar

syzlang is a domain-specific declarative language used to explicitly model the target kernel application binary interface. A syzlang definition contains specifications for every supported system call, including formal parameter types, structure layouts, union variants, and valid integer ranges or flag enumerations. It also supports the concept of resources, which explicitly defines data dependencies across independent system calls: for example, a system call like `open` or `socket` returns a specific resource handle (a file descriptor) and the syzlang grammar specifies which subsequent calls, (e.g. `read`, `write`, `bind`, or `ioctl`) require that exact resource type as an input argument.

2.2 Fuzzing process

The fuzzing loop operates as an evolutionary system driven by the coverage metrics:

1. syz-manager selects a baseline program from its corpus or decides to generate a new program sequence based on the available syzlang definitions;
2. the fuzzer modifies the program by inserting new system calls, deleting existing ones, changing flag values, or mutating data buffers embedded within pointer structures;
3. the mutated call sequence is serialized and sent to syz-executor;
4. the executor unpacks the data structures, executes the system calls in sequence, and catches any non-fatal flags or error codes. During execution, the kernel's KCOV subsystem records every basic block address touched by the process;
5. the executor passes the traces back to syz-manager and if a new edge transition is detected, the fuzzer initiates an inline minimization routine to strip away redundant operations while preserving the exact code coverage path;
6. after minimization, the manager updates the master corpus and distributes the updated input to all other active virtual machine workers.

When a fatal condition occurs and the kernel crashes, Syzkaller transitions to crash triaging and automated reproduction to try to generate a minimal program that reproduces the bug:

1. when the kernel panics, syz-manager extracts the logs from the crashed machine and kills it;
2. the manager then extracts the call stack trace and matches it against an internal database of known unique vulnerabilities to prevent duplicating the same bug over and over;
3. the manager invokes the syz-repro reproduction utility on a new VM instance;
4. if the reproducer manages to find the right, minimal sequence of system calls, it produces a minimal reproduction program in syzlang, then using the utility `syz-prog2c` it gets compiled to C. The C program can finally be compiled to an executable that reproduces the original bug. If syz-repro cannot find a suitable reproduction program, it flags the bug as *non-reproducible*.

To enable continuous, 24/7 testing, Syzkaller was deployed on a private server on a dedicated VM¹ with 16 cores, 32 GB of RAM and 16 GB of ZRAM swap to quickly handle the kernel recompilation and to be able to navigate the Syzkaller dashboard which is resource intensive². This configuration supports 8 concurrent VMs, each allocated 2 GB of RAM and 2 vCPUs, which translates to about 120/140 execs/s.

Syzkaller has to be installed from source, in particular the steps to follow are:

1. install the dependencies: `make gcc flex bison libncurses-dev libelf-dev libssl-dev qemu-system-x86 git debootstrap`;
2. install Docker and the Golang toolchain;
3. clone the [Syzkaller repository](#) and run the `make` command using the `syz-env` prefix to set up the build environment;
4. clone the [Linux repository](#) and apply the base configuration with the commands `make defconfig` and `make kvm_quest.config`, since it will be executed inside a virtual machine;
5. apply the following flags and compile the kernel.

```
./scripts/config --enable CONFIG_KASAN
./scripts/config --enable CONFIG_KASAN_INLINE
./scripts/config --enable CONFIG_UBSAN
./scripts/config --enable CONFIG_KASAN_GENERIC
./scripts/config --enable CONFIG_KCOV
./scripts/config --enable CONFIG_KCOV_ENABLE_COMPARISONS
./scripts/config --enable CONFIG_DEBUG_INFO
./scripts/config --enable CONFIG_DEBUG_INFO_DWARF4
./scripts/config --enable CONFIG_CONFIGFS_FS
./scripts/config --enable CONFIG_SECURITYFS
```

Now that the kernel is built, Syzkaller needs a virtual machine where to inject the kernel and boot into to start the fuzzing. Luckily, Syzkaller provides a tool (`create-image.sh`) to download and prepare a minimal Debian image and create the SSH key pair to connect to the VM.

This is the standard approach, but, especially for slower hardware, running a fully fledged distribution, even if minimal, could be too slow, especially regarding the boot time, as every crash caused by Syzkaller is followed by a reboot of a VM. To solve this problem and speed up the fuzzing process, instead of a Debian image, a [buildroot](#) image has been used.

3.1 Configuration

Syzkaller has been configured to check only a subset of the available system calls, otherwise the fuzzing process would have required many days³ to find the bugs introduced, keeping the server busy⁴ targeting all the modules of the Linux kernel.

¹Nested virtualization is supported by the host.

²Navigating the coverage page needs Syzkaller to load the whole database in RAM.

³To find all the bugs a couple of days are still required.

⁴The host server is also used for other tasks.

As we can see from the configuration below, the calls enabled are the buggy system calls introduced plus the ones needed to execute the patched functions that will be presented in the next chapter.

```
{
  "target": "linux/amd64",
  "http": "0.0.0.0:56741",
  "workdir": "/home/syzkaller/project/syzkaller/workdir",
  "kernel_obj": "/home/syzkaller/project/linux",
  "image": "/home/syzkaller/image/buildroot/output/images/rootfs.ext4",
  "sshkey": "/home/syzkaller/image/trixie.id_rsa",
  "syzkaller": "/home/syzkaller/project/syzkaller",
  "procs": 4,
  "type": "qemu",
  "enable_syscalls": [
    "ictra_uaf",
    "ictra_heap_oob",
    "ictra_static_oob",
    "open",
    "openat",
    "read",
    "write",
    "socket",
    "setsockopt",
    "openat$dvkm",
    "ioctl$DVKM_INTEGER_OVERFLOW",
    "ioctl$DVKM_INTEGER_UNDERFLOW",
    "ioctl$DVKM_STACK_BUFFER_OVERFLOW",
    "ioctl$DVKM_HEAP_BUFFER_OVERFLOW",
    "ioctl$DVKM_STACK_OOBR",
    "ioctl$DVKM_STACK_OOBW",
    "ioctl$DVKM_HEAP_OOBR",
    "ioctl$DVKM_HEAP_OOBW",
    "ioctl$DVKM_USE_AFTER_FREE",
    "ioctl$DVKM_DOUBLE_FREE"
  ],
  "vm": {
    "count": 8,
    "kernel": "/home/syzkaller/project/linux/arch/x86/boot/bzImage",
    "cmdline": "net.ifnames=0 root=/dev/sda console=ttyS0 selinux=0
mitigations=off panic=1",
    "cpu": 2,
    "mem": 2048
  }
}
```

3.1.1 Buildroot

As said before, an optimization introduced to speed up the fuzzing process is the use of [buildroot](#), a very minimal distribution image, which allows for quick startup times⁵ during fuzzing and bug reproductions.

⁵Depending on the host load and the disk throughput, boot times can drop from minutes to seconds.

Once cloned the repository, the image can be configured via `make menuconfig` to select the programs, libraries and toolchains to include in the final bootable `rootfs.ext2` disk image. To setup and compile the image, the only configuration needed is to enable `glibc` as the standard C library via `make menuconfig` and then compile normally. Once compiled, the image is still not ready to be used by Syzkaller as it misses a crucial `inittab` configuration: when the system boots it needs to mount `sysfs` and `debugfs` to allow Syzkaller to gather KCOV information and interact with the kernel. To do so, it is sufficient to mount the final image as a loop device or via `debugfs` to modify the `/etc/inittab` file and add the missing configuration.

To demonstrate Syzkaller's detection capabilities, some bugs have been introduced into the Linux kernel source code, such as completely new broken system calls, bugs in already existing system calls and a vulnerable kernel module (DVKM).

4.1 Custom Syscalls

The first system call is named `ictra_uaf` which takes two arguments and triggers a use after free bug when the magic values `0xCAFECAFE` and `0xDEADBEEF` are passed respectively as first and second argument:

```
SYSCALL_DEFINE2(ictra_uaf, unsigned long, cmd, unsigned long, val)
{
    char *ptr;
    if (cmd != 0xCAFECAFE || val != 0xDEADBEEF)
        return -EINVAL;

    ptr = kmalloc(16, GFP_KERNEL);
    if (!ptr)
        return -ENOMEM;

    kfree(ptr);
    ptr = 'A';
    return 0;
}
```

The second system call is `ictra_heap_oob`, which takes two parameters and behaves as follows:

- when `cmd` is not equal to `0xBAADF00D` it returns an error;
- when `cmd` is equal to `0xBAADF00D` it allocates a 16 bytes buffer and then calls `memset` on that buffer for `size` bytes. Because the if guard checks whether `size <= 32`, if `17 <= size <= 32`, the system call access an out of bound index on `ptr`.

```
SYSCALL_DEFINE2(ictra_heap_oob, unsigned long, cmd, unsigned long, size)
{
    char *ptr;
    if (cmd != 0xBAADF00D)
        return -EINVAL;
    ptr = kmalloc(16, GFP_KERNEL);
    if (!ptr)
        return -ENOMEM;
    if (size <= 32) {
        memset(ptr, 'B', size);
    }
    kfree(ptr);
    return 0;
}
```

The last system call, `ictra_static_oob`, behaves very similar to the previous one, but the buffer is allocated in the static part of the memory and the index accessed is not bound or checked in any way.

```
static char global_buffer[16];
SYSCALL_DEFINE2(ictra_static_oob, unsigned long, cmd, unsigned long, index)
{
    if (cmd != 0xFEEDFACE)
        return -EINVAL;

    global_buffer[index] = 'C';
    return 0;
}
```

4.2 Kernel patches

To stress test the fuzzer, some functions have been patched⁶ to trigger a bugs when some of the parameters are set to a certain value.

The next block of code shows the patch applied to the `vfs_write` function under the `fs` module.

```
diff --git a/fs/read_write.c b/fs/read_write.c
index 50bff7edc..84d068d04 100644
--- a/fs/read_write.c
+++ b/fs/read_write.c
@@ -679,6 +679,16 @@ ssize_t vfs_write(struct file *file, const char __user
 *buf, size_t count, loff_
     ret = rw_verify_area(WRITE, file, pos, count);
     if (ret)
         return ret;
+
+     if (count == 4919) {
+         char *bug_ptr = kmalloc(16, GFP_KERNEL);
+         if (bug_ptr) {
+             kfree(bug_ptr);
+             bug_ptr = 'X';
+         }
+     }
+
     if (count > MAX_RW_COUNT)
         count = MAX_RW_COUNT;
     file_start_write(file);
```

As we can see, when the magic count value is set to 4919 (0x1337), the write function triggers a UAF bug.

⁶For reproducibility the patches have been applied using git.

Finally, another patch has been applied to `do_ip_setsockopt`:

```
diff --git a/net/ipv4/ip_sockglue.c b/net/ipv4/ip_sockglue.c
index a55ef327e..ccf657eb9 100644
--- a/net/ipv4/ip_sockglue.c
+++ b/net/ipv4/ip_sockglue.c
@@ -897,6 +897,15 @@ int do_ip_setsockopt(struct sock *sk, int level, int
optname,
    int val = 0, err, retv;
    bool needs_rtnl = setsockopt_needs_rtnl(optname);

+    if (optname == IP_OPTIONS && optlen > 0 && optlen < 10) {
+        char *dummy_ptr = kmalloc(optlen, GFP_KERNEL);
+        if (dummy_ptr) {
+            dummy_ptr[optlen] = 'A';
+            kfree(dummy_ptr);
+        }
+    }
+
    switch (optname) {
    case IP_PKTINFO:
    case IP_RECVTTL:
```

When the function is called with an `optlen` between 1 and 9, an out-of-bound bug is triggered.

4.3 DVKM

To further demonstrate the capabilities of Syzkaller, a vulnerable kernel module ([DVKM](#)) has use.

From the original project README:

This example kernel module is designed to facilitate fuzzing with syzkaller, an efficient kernel fuzzer. It incorporates deliberately vulnerable code that showcases various types of security vulnerabilities, such as integer overflow, integer underflow, use-after-free, double free, stack and heap overflows, and out-of-bounds read/write scenarios.

The module has been derived from a previous project of mine, the “Damn Vulnerable C Program” (available at [https://github.com/hardik05/Damn_Vulnerable_C_Program]), which was crafted for the purpose of understanding and practicing user-mode fuzzing techniques. With this kernel module, I intend to provide a learning platform for those interested in exploring kernel-mode fuzzing.

By utilizing this module as a testbed, security researchers and enthusiasts can gain hands-on experience in identifying and mitigating these critical vulnerabilities. It serves as a valuable resource for understanding the inner workings of security flaws in kernel code and devising effective ways to enhance system robustness.

This module has been included into the source code under `lib/`, adding the object to the Makefile (`obj-y += dvkm.o`) and then rebuilding the kernel.

The vulnerabilities contained in DVKM are:

- integer underflow/overflow;
- heap/stack overflow;
- heap/stack out-of-bound read/write;
- use-after-free;
- double free.

To enable Syzkaller to target the library the following description has been defined:

```
resource fd_dvkm[fd]

openat$dvkm(fd const[AT_FDCWD], file ptr[in, string["/proc/dvkm"]], flags
const[0_RDWR], mode const[0]) fd_dvkm

dvkm_obj {
    width    int32
    height   int32
    datasize  bytesize[data, int32]
    pad       const[0, int32]
    data      ptr[in, array[int8]]
}

dvkm_obj_intops {
    width    int32[1:0x7fffffff]
    height   int32[1:0x7fffffff]
    datasize  bytesize[data, int32]
    pad       const[0, int32]
    data      ptr[in, array[int8]]
}

# 0xC0184400 – INTEGER OVERFLOW
ioctl$DVKM_INTEGER_OVERFLOW(fd fd_dvkm, cmd const[0xC0184400], arg ptr[in,
dvkm_obj_intops])

# 0xC0184401 – INTEGER UNDERFLOW
ioctl$DVKM_INTEGER_UNDERFLOW(fd fd_dvkm, cmd const[0xC0184401], arg ptr[in,
dvkm_obj_intops])

# 0xC0184402 – STACK BUFFER OVERFLOW
ioctl$DVKM_STACK_BUFFER_OVERFLOW(fd fd_dvkm, cmd const[0xC0184402], arg ptr[in,
dvkm_obj])

# 0xC0184403 – HEAP BUFFER OVERFLOW
ioctl$DVKM_HEAP_BUFFER_OVERFLOW(fd fd_dvkm, cmd const[0xC0184403], arg ptr[in,
dvkm_obj])

# 0xC0184405 – STACK OUT-OF-BOUNDS READ
ioctl$DVKM_STACK_OOBR(fd fd_dvkm, cmd const[0xC0184405], arg ptr[in, dvkm_obj])

# 0xC0184406 – STACK OUT-OF-BOUNDS WRITE
ioctl$DVKM_STACK_OOBW(fd fd_dvkm, cmd const[0xC0184406], arg ptr[in, dvkm_obj])
```

```
# 0xC0184407 – HEAP OUT-OF-BOUNDS READ
ioctl$DVKM_HEAP_OOBR(fd fd_dvkm, cmd const[0xC0184407], arg ptr[in, dvkm_obj])

# 0xC0184408 – HEAP OUT-OF-BOUNDS WRITE
ioctl$DVKM_HEAP_OOBW(fd fd_dvkm, cmd const[0xC0184408], arg ptr[in, dvkm_obj])

# 0xC018440A – USE-AFTER-FREE
ioctl$DVKM_USE_AFTER_FREE(fd fd_dvkm, cmd const[0xC018440A], arg ptr[in,
dvkm_obj])

# 0xC018440B – DOUBLE FREE
ioctl$DVKM_DOUBLE_FREE(fd fd_dvkm, cmd const[0xC018440B], arg ptr[in,
dvkm_obj])
```

As we can see, once the vulnerable resource is opened, the fuzzer can target each vulnerability using `ioctl` with a specific value.

CHAPTER 5

DEMONSTRATION OF FINDINGS

To validate that Syzkaller successfully identified the injected bugs, we utilized the C code generated by `syz-prog2c`.

To automatically discover and execute reproductions, the script `scripts/run_repro.sh` parses the Syzkaller working directory, looking for `repro.cprog` files under the `crashes` directory. Once found, it asks which repro to execute:

1. the reproduction is compiled using the `-static` flag to ensure the binary runs without external library dependencies inside the minimal buildroot environment and then it is copied mounting the image in a loop device:

```
gcc -static -O0 -g -o repro repro.c -lpthread -ldl
mount -o loop ~/image/buildroot/output/images/rootfs_repro.ext4 /mnt/
buildroot
cp repro /mnt/buildroot/root/repro
umount /mnt/buildroot
```

2. then, a VM is started using the image containing the reproduction so that we can connect to it and launch the reproduction binary:

```
qemu-system-x86_64 \
-M pc \
-kernel "${PROJECT_DIR}/linux/arch/x86/boot/bzImage" \
-drive file=/home/syzkaller/image/buildroot/output/images/
rootfs_repro.ext4,if=virtio,format=raw \
-append "net.ifnames=0 root=/dev/vda console=ttyS0 selinux=0
mitigations=off panic=1" \
-net nic,model=virtio \
-net user,hostfwd=tcp::11022-:22 \
-nographic \
-cpu host \
-smp 2 \
-m 2048 \
-enable-kvm 2>&1 | tee /tmp/repro_output.log
```

3. at this point we can login with the `root` username (and no password) and launch the executable (`./repro`).

5.1 Reading reproduction outputs

In this subsection will be presented some reproduction outputs from different memory corruption bugs to then analyze the information provided.

ictra_uaf:

```
=====
BUG: KASAN: slab-use-after-free in __x64_sys_ictra_uaf+0x11e/0x150
Write of size 1 at addr ffff88800338ca80 by task repro/145

CPU: 0 UID: 0 PID: 145 Comm: repro Not tainted 7.1.0-rc5-ge8c2f9fdadee-dirty #8
PREEMPT(lazy)
Hardware name: QEMU Standard PC (i440FX + PIIX, 1996), BIOS 1.16.3-
debian-1.16.3-2 04/01/2014

Call Trace:
<TASK>
dump_stack_lvl+0x97/0xe0
print_report+0x157/0x4c9
? __pfx__raw_spin_lock_irqsave+0x10/0x10
? __x64_sys_ictra_uaf+0x11e/0x150
kasan_report+0xce/0x100
? __x64_sys_ictra_uaf+0x11e/0x150
__x64_sys_ictra_uaf+0x11e/0x150
do_syscall_64+0xee/0x590
entry_SYSCALL_64_after_hwframe+0x77/0x7f
RIP: 0033:0x411af9
Code: c0 79 93 eb d5 48 8d 7c 1d 00 eb 99 0f 1f 44 00 00 48 89 f8 48 89 f7 48
89 d6 48 89 ca 4d 89 c2 4d 89 c8 4c 8b 4c 24 08 0f 8
RSP: 002b:00007ffcl804f28 EFLAGS: 00000206 ORIG_RAX: 00000000000001d8
RAX: ffffffffda RBX: 00007ffcl805058 RCX: 0000000000411af9
RDX: 0000000000409752 RSI: 00000000deadbeef RDI: 00000000cafe0000
RBP: 00007ffcl804f30 R08: 0000000000000000 R09: 0000000000000001
R10: 0000000000000000 R11: 0000000000000206 R12: 00007ffcl805048
R13: 0000000000000002 R14: 000000000049ee40 R15: 0000000000000002
</TASK>

Allocated by task 145:
kasan_save_stack+0x30/0x50
kasan_save_track+0x14/0x30
__kasan_kmalloc+0x8f/0xa0
__x64_sys_ictra_uaf+0xc7/0x150
do_syscall_64+0xee/0x590
entry_SYSCALL_64_after_hwframe+0x77/0x7f

Freed by task 145:
kasan_save_stack+0x30/0x50
kasan_save_track+0x14/0x30
kasan_save_free_info+0x3b/0x70
__kasan_slab_free+0x43/0x70
kfree+0x149/0x3a0
__x64_sys_ictra_uaf+0xdc/0x150
do_syscall_64+0xee/0x590
```



```

entry_SYSCALL_64_after_hwframe+0x77/0x7f
The buggy address belongs to the object at ffff88800338ca80
  which belongs to the cache kmalloc-16 of size 16
The buggy address is located 0 bytes inside of
  freed 16-byte region [ffff88800338ca80, ffff88800338ca90)

The buggy address belongs to the physical page:
page: refcount:0 mapcount:0 mapping:0000000000000000 index:0x0 pfn:0x338c
flags: 0x1000000000000000(node=0|zone=1)
page_type: f5(slab)
raw: 0100000000000000 ffff888001041640 dead000000000100 dead000000000122
raw: 0000000000000000 0000000000800080 00000000f5000000 0000000000000000
page dumped because: kasan: bad access detected

Memory state around the buggy address:
ffff88800338c980: fc fc fc fc fc fc fc fc fc fc fc fc fc fc fc
ffff88800338ca00: fc fc fc fc fc fc fc fc fc fc fc fc fc fc fc
>ffff88800338ca80: fa fb fc fc fa fb fc fc 00 00 fc fc fa fb fc fc
                    ^
ffff88800338cb00: 00 00 fc fc fa fb fc fc fa fb fc fc fa fb fc fc
ffff88800338cb80: fa fb fc fc fa fb fc fc 00 00 fc fc 00 00 fc fc
=====
Disabling lock debugging due to kernel taint

```

do_ip_setsockopt:

```

=====
BUG: KASAN: slab-out-of-bounds in do_ip_setsockopt+0x301d/0x30e0
Write of size 1 at addr ffff888002147589 by task repro/144

CPU: 1 UID: 0 PID: 144 Comm: repro Not tainted 7.1.0-rc5-ge8c2f9fdadee-dirty #8
PREEMPT(lazy)
Hardware name: QEMU Standard PC (i440FX + PIIX, 1996), BIOS 1.16.3-
debian-1.16.3-2 04/01/2014
Call Trace:
<TASK>
dump_stack_lvl+0x97/0xe0
print_report+0x157/0x4c9
? __pfx__raw_spin_lock_irqsave+0x10/0x10
? do_ip_setsockopt+0x301d/0x30e0
kasan_report+0xce/0x100
? do_ip_setsockopt+0x301d/0x30e0
do_ip_setsockopt+0x301d/0x30e0
? __pfx_do_ip_setsockopt+0x10/0x10
? __handle_mm_fault+0x6e8/0x2710
? __pfx__handle_mm_fault+0x10/0x10
ip_setsockopt+0x5a/0xf0
tcp_setsockopt+0xa7/0x100
? __pfx_sock_common_setsockopt+0x10/0x10
do_sock_setsockopt+0xf3/0x1d0
__sys_setsockopt+0x119/0x190

```

```

__x64_sys_setsockopt+0xbd/0x160
? fpregs_assert_state_consistent+0xcd/0x150
do_syscall_64+0xee/0x590
entry_SYSCALL_64_after_hwframe+0x77/0x7f
RIP: 0033:0x411b79
Code: c0 79 93 eb d5 48 8d 7c 1d 00 eb 99 0f 1f 44 00 00 48 89 f8 48 89 f7 48
89 d6 48 89 ca 4d 89 c2 4d 89 c8 4c 8b 4c 24 08 0f 8
RSP: 002b:00007ffec0e95aa8 EFLAGS: 00000202 ORIG_RAX: 0000000000000036
RAX: ffffffffda RBX: 00007ffec0e95be8 RCX: 0000000000411b79
RDX: 0000000000000004 RSI: 0000000000000000 RDI: 0000000000000003
RBP: 00007ffec0e95ac0 R08: 0000000000000009 R09: 00000000004aa278
R10: 0000200000000000 R11: 0000000000000202 R12: 00007ffec0e95bd8
R13: 0000000000000002 R14: 000000000049ee40 R15: 0000000000000002
</TASK>

```

Allocated by task 144:

```

kasan_save_stack+0x30/0x50
kasan_save_track+0x14/0x30
__kasan_kmalloc+0x8f/0xa0
__kmalloc_noprof+0x19d/0x500
do_ip_setsockopt+0x491/0x30e0
ip_setsockopt+0x5a/0xf0
tcp_setsockopt+0xa7/0x100
do_sock_setsockopt+0xf3/0x1d0
__sys_setsockopt+0x119/0x190
__x64_sys_setsockopt+0xbd/0x160
do_syscall_64+0xee/0x590
entry_SYSCALL_64_after_hwframe+0x77/0x7f
The buggy address belongs to the object at ffff888002147580
which belongs to the cache kmalloc-16 of size 16
The buggy address is located 0 bytes to the right of
allocated 9-byte region [ffff888002147580, ffff888002147589)

```

The buggy address belongs to the physical page:

```

page: refcount:0 mapcount:0 mapping:0000000000000000 index:0xffff888002147880
pfn:0x2147
flags: 0x1000000000000200(workingset|node=0|zone=1)
page_type: f5(slab)
raw: 0100000000000200 ffff888001041640 ffff888001040290 ffff888001040290
raw: ffff888002147880 0000000000800044 00000000f5000000 0000000000000000
page dumped because: kasan: bad access detected

```

Memory state around the buggy address:

```

ffff888002147480: fc fc fc fc fc fc fc fc fc fc fc fc fc fc fc
ffff888002147500: fc fc fc fc fc fc fc fc fc fc fc fc fc fc fc
>ffff888002147580: 00 01 fc fc fa fb fc fc 00 00 fc fc fa fb fc fc
^
ffff888002147600: fa fb fc fc 00 00 fc fc 00 00 fc fc fa fb fc fc
ffff888002147680: 00 05 fc fc fa fb fc fc fa fb fc fc 00 01 fc fc
=====

```

Disabling lock debugging due to kernel taint

ictra_static_oob:

```
UBSAN: array-index-out-of-bounds in kernel/sys.c:3117:15
index 4294967297 is out of range for type 'char [16]'
CPU: 1 UID: 0 PID: 145 Comm: repro Not tainted 7.1.0-rc5-ge8c2f9fdadee-dirty #9
PREEMPT(lazy)
Hardware name: QEMU Standard PC (i440FX + PIIX, 1996), BIOS 1.16.3-
debian-1.16.3-2 04/01/2014
Call Trace:
<TASK>
dump_stack_lvl+0x97/0xe0
ubsan_epilogue+0xa/0x30
__ubsan_handle_out_of_bounds+0xcc/0xf0
__x64_sys_ictra_static_oob+0x90/0xb0
do_syscall_64+0xee/0x590
entry_SYSCALL_64_after_hwframe+0x77/0x7f
RIP: 0033:0x411af9
Code: c0 79 93 eb d5 48 8d 7c 1d 00 eb 99 0f 1f 44 00 00 48 89 f8 48 89 f7 48
89 d6 48 89 ca 4d 89 c2 4d 89 c8 4c 8b 4c 24 08 0f 05 <48> 3d 01 f0 ff ff 73 8
RSP: 002b:00007fff0a493d88 EFLAGS: 00000206 ORIG_RAX: 00000000000001da
RAX: ffffffff0a493eb8 RBX: 00007fff0a493eb8 RCX: 0000000000411af9
RDX: 0000000000409752 RSI: 0000000100000001 RDI: 00000000feedface
RBP: 00007fff0a493d90 R08: 0000000000000000 R09: 0000000000000001
R10: 0000000000000000 R11: 0000000000000206 R12: 00007fff0a493ea8
R13: 0000000000000002 R14: 000000000049ee40 R15: 0000000000000002
</TASK>
```

5.1.1 KASAN output

Every report starts with a line of the form:

```
BUG: KASAN: slab-out-of-bounds in do_ip_setsockopt+0x301d/0x30e0
BUG: KASAN: slab-use-after-free in __x64_sys_ictra_uaf+0x11e/0x150
```

The pieces of information contained in this line are:

- **Bug class:** slab-out-of-bounds means the access went outside the boundaries of a live slab object. slab-use-after-free means the memory had been returned to the allocator before it was accessed again.
- **Function and offset:** The function+0x0FFSET/0xSIZE field is the instruction that triggered KASAN. do_ip_setsockopt+0x301d/0x30e0 means the bad instruction is at byte offset 0x301d inside a function that is 0x30e0 bytes long.

The next line gives us the **direction** of the bug:

```
Write of size 1 at addr ffff888002147589 by task repro/144
Write of size 1 at addr ffff88800338ca80 by task repro/145
```

- **Direction:** Read or Write.
- **Size:** The number of bytes in the bad access.
- **Address:** The exact kernel virtual address that was touched.
- **Task:** The process name and PID (repro/144 and repro/145, the reproduction binaries).

Then, we have the **task context**:

```
CPU: 1 UID: 0 PID: 144 Comm: repro Not tainted 7.1.0-rc5-ge8c2f9fdadee-dirty #8
```

- **CPU:** Which logical core was running at the moment of the violation, useful when the bug is related to concurrency issues.
- **UID / PID.**
- **Comm:** The comm field is the thread name, truncated to 15 characters.
- **Not tainted:** The kernel does not contain out-of-tree modules.
- **Kernel version:** The string 7.1.0-rc5-ge8c2f9fdadee-dirty is a git describe output, where the dirty suffix means there were uncommitted local changes⁷.

The next lines contain the **call trace**, useful to reconstruct which calls triggered the bug:

```
Call Trace:
<TASK>
  dump_stack_lvl+0x97/0xe0
  print_report+0x157/0x4c9
  ? __pfx__raw_spin_lock_irqsave+0x10/0x10
  ? do_ip_setsockopt+0x301d/0x30e0
  kasan_report+0xce/0x100
  ? do_ip_setsockopt+0x301d/0x30e0
  do_ip_setsockopt+0x301d/0x30e0
```

⁷The bugs introduced in the project.

```

ip_setsockopt+0x5a/0xf0
tcp_setsockopt+0xa7/0x100
do_sock_setsockopt+0xf3/0x1d0
__sys_setsockopt+0x119/0x190
__x64_sys_setsockopt+0xbd/0x160
do_syscall_64+0xee/0x590
entry_SYSCALL_64_after_hwframe+0x77/0x7f
</TASK>

```

The trace is printed top-to-bottom with the most recent frame first, so to read it chronologically, the direction is bottom-to-top.

Another important piece of information is the **register dump** at the moment the system call is performed. On x86-64, the Linux syscall calling convention maps arguments to registers in this fixed order: RDI (arg1), RSI (arg2), RDX (arg3), R10 (arg4), R08 (arg5), R09 (arg6). ORIG_RAX holds the syscall number as saved by the kernel before it overwrites RAX with the return value.

slab-out-of-bounds: setsockopt call

```

ORIG_RAX: 0000000000000036
RAX: ffffffffda RBX: 00007ffec0e95be8 RCX: 0000000000411b79
RDX: 0000000000000004 RSI: 0000000000000000 RDI: 0000000000000003
RBP: 00007ffec0e95ac0 R08: 0000000000000009 R09: 00000000004aa278
R10: 0000200000000000 R11: 0000000000000202 R12: 00007ffec0e95bd8

```

- **ORIG_RAX = 0x36.** Decimal 54. This is `sys_setsockopt` on x86-64.
- **RDI = 0x3.** First argument, the socket file descriptor.
- **RSI = 0x0.** Second argument, the socket level. 0 is `IPPROTO_IP`.
- **RDX = 0x4.** Third argument, the option name. `IP_OPTIONS` is option 4 at level `IPPROTO_IP`, instructing the function to copy a raw byte buffer directly into the IP options field of the socket.
- **R10 = 0x0000200000000000.** Fourth argument, the `optval` pointer.
- **R08 = 0x9.** Fifth argument, `optlen`, the number of bytes to copy, the option that triggered the bug introduced before.

slab-use-after-free: ictra_uaf call

```

ORIG_RAX: 00000000000001d8
RAX: ffffffffda RBX: 00007ffc1a805058 RCX: 0000000000411af9
RDX: 0000000000409752 RSI: 00000000deadbeef RDI: 00000000cafecafe
RBP: 00007ffc1a804f30 R08: 0000000000000000 R09: 0000000000000001
R10: 0000000000000000 R11: 0000000000000206 R12: 00007ffc1a805048

```

- **ORIG_RAX = 0x1d8.** Decimal 472, the custom syscall number at which the `ictra_uaf` syscall has been registered.
- **RDI = 0x00000000cafecafe.** First argument, `0xcafecafe`, the first magic value.
- **RSI = 0x00000000deadbeef.** Second argument, `0xdeadbeef`, the second magic value.

Now, we have the **allocation trace**, which tells us where the corrupted memory was initially allocated:

```
Allocated by task 144:
kasan_save_stack+0x30/0x50
kasan_save_track+0x14/0x30
__kasan_kmalloc+0x8f/0xa0
__kmalloc_noprof+0x19d/0x500
do_ip_setsockopt+0x491/0x30e0
ip_setsockopt+0x5a/0xf0
```

When a UAF bug occurs, we also have the **free trace**, which tells us who freed the later accessed buffer:

```
Freed by task 145:
kasan_save_stack+0x30/0x50
kasan_save_track+0x14/0x30
kasan_save_free_info+0x3b/0x70
__kasan_slab_free+0x43/0x70
kfree+0x149/0x3a0
__x64_sys_ictra_uaf+0xdc/0x150
do_syscall_64+0xee/0x590
entry_SYSCALL_64_after_hwframe+0x77/0x7f
```

Finally we have the **memory state map**:

do_ip_setsockopt:

```
Memory state around the buggy address:
ffff888002147480: fc fc fc fc fc fc fc fc fc fc fc fc fc fc fc
ffff888002147500: fc fc fc fc fc fc fc fc fc fc fc fc fc fc fc
>ffff888002147580: 00 01 fc fc fa fb fc fc 00 00 fc fc fa fb fc fc
               ^
ffff888002147600: fa fb fc fc 00 00 fc fc 00 00 fc fc fa fb fc fc
```

ictra_uaf:

```
Memory state around the buggy address:
ffff88800338c980: fc fc fc fc fc fc fc fc fc fc fc fc fc fc fc
ffff88800338ca00: fc fc fc fc fc fc fc fc fc fc fc fc fc fc fc
>ffff88800338ca80: fa fb fc fc fa fb fc fc 00 00 fc fc fa fb fc fc
               ^
ffff88800338cb00: 00 00 fc fc fa fb fc fc fa fb fc fc fa fb fc fc
ffff88800338cb80: fa fb fc fc fa fb fc fc 00 00 fc fc 00 00 fc fc
```

This is a hex dump of the KASAN *shadow memory*. Each shadow byte encodes the state of 8 consecutive bytes of real memory. The line prefixed with > contains the buggy address, while the ^ caret marks the shadow byte that covers the bad access.

Each address on the left is the real kernel address of the first byte in that row's span. Because each shadow byte covers 8 real bytes, a row of 16 shadow bytes covers 128 bytes of heap.

Shadow value	Meaning
00	All 8 real bytes are fully accessible.
01 to 07	Only the first N bytes of the 8-byte group are accessible. The rest are redzone. This value appears at the tail of an allocation that does not end on an 8-byte boundary.
fa	Slab left redzone. These bytes precede the allocated object within the slab slot, or mark a freed object in quarantine.
fb	Slab right redzone or freed quarantine metadata. These bytes follow the live object or mark it as pending free.
fc	Freed slab memory. The entire 8-byte group belongs to memory that has been returned to the slab and is fully off-limits.

Table 1: Shadow byte values

In the out-of-bounds example, the object starts at `ffff888002147580`. Shadow byte 0 is `00`: all 8 bytes at offsets 0-7 are valid. Shadow byte 1 is `01`: only offset 8 is accessible; offsets 9-15 are not. The bad address `ffff888002147589` is at offset 9, which falls inside the `01` group but past its single accessible byte. The caret sits under `01`, confirming the exact boundary crossing.

In the UAF example, the object starts at `ffff88800338ca80`. Shadow byte 0 is `fa`, meaning the first 8 bytes are already marked as freed or redzone. The caret sits under `fa`. Any access here is a use-after-free regardless of which byte within the group was touched.

5.1.2 UBSAN output

Differently from KASAN, UBSAN detects the C undefined behavior, such as, in our example, an out-of-bound access to an array not declared on the heap. In this case the compiler emitted bounds checking code for the array of known size declared before the `ictra_static_oob` syscall to catch attempts to access an index past the end of the array.

As before, in the output the first lines describe the type of bug, but here we also have the exact location in the code where the bug happened:

```
UBSAN: array-index-out-of-bounds in kernel/sys.c:3117:15
index 4294967297 is out of range for type 'char [16]'
```

The **call trace** is still similar to the previous outputs:

```
dump_stack_lvl+0x97/0xe0
ubsan_epilogue+0xa/0x30
__ubsan_handle_out_of_bounds+0xcc/0xf0
__x64_sys_ictra_static_oob+0x90/0xb0
do_syscall_64+0xee/0x590
entry_SYSCALL_64_after_hwframe+0x77/0x7f
```

As well as the registers dump:

```
ORIG_RAX: 000000000000001da
RAX: ffffffffda  RBX: 00007fff0a493eb8  RCX: 0000000000411af9
RDX: 0000000000409752  RSI: 0000000100000001  RDI: 00000000feedface
RBP: 00007fff0a493d90  R08: 0000000000000000  R09: 0000000000000001
R10: 0000000000000000  R11: 0000000000000206  R12: 00007fff0a493ea8
```

- **ORIG_RAX = 0x1da.** Decimal 474, the custom syscall number at which the `ictra_uaf` syscall has been registered.
- **RDI = 0x00000000feedface.** First argument, the magic value to trigger the bug.
- **RSI = 0x0000000100000001.** Second argument, the index to access, which is not checked before attempting to access the original array.